

Context 备份

目录

- 一、需求
 - 1.1、pten对DeviceContext的需求
 - 1.2、训练和推理重构对DeviceContext的需求
 - 1.2.1 多个Context复用cudnn workspace场景（或复用其它资源）。
 - 1.2.2、kernel级别支持IO和exec stream的并发。
 - 1.2.3、支持外部用户传流（IDG业务强需求），进一步拓展，支持外部用户传资源（cublasHandle, cudnnHandle）。
- 二、方案
 - 2.1 方案综述
 - 2.2、现口
 - 2.2.1、现有接口
 - 2.2.2、为支持预测场景，需新增的接口
- 三、方案论证
 - 3.1、兼容fluid框架
 - 3.2、对pten长久状态支持
 - 3.3、开发和兼容性代价



一、需求

1.1、pten对DeviceContext的需求

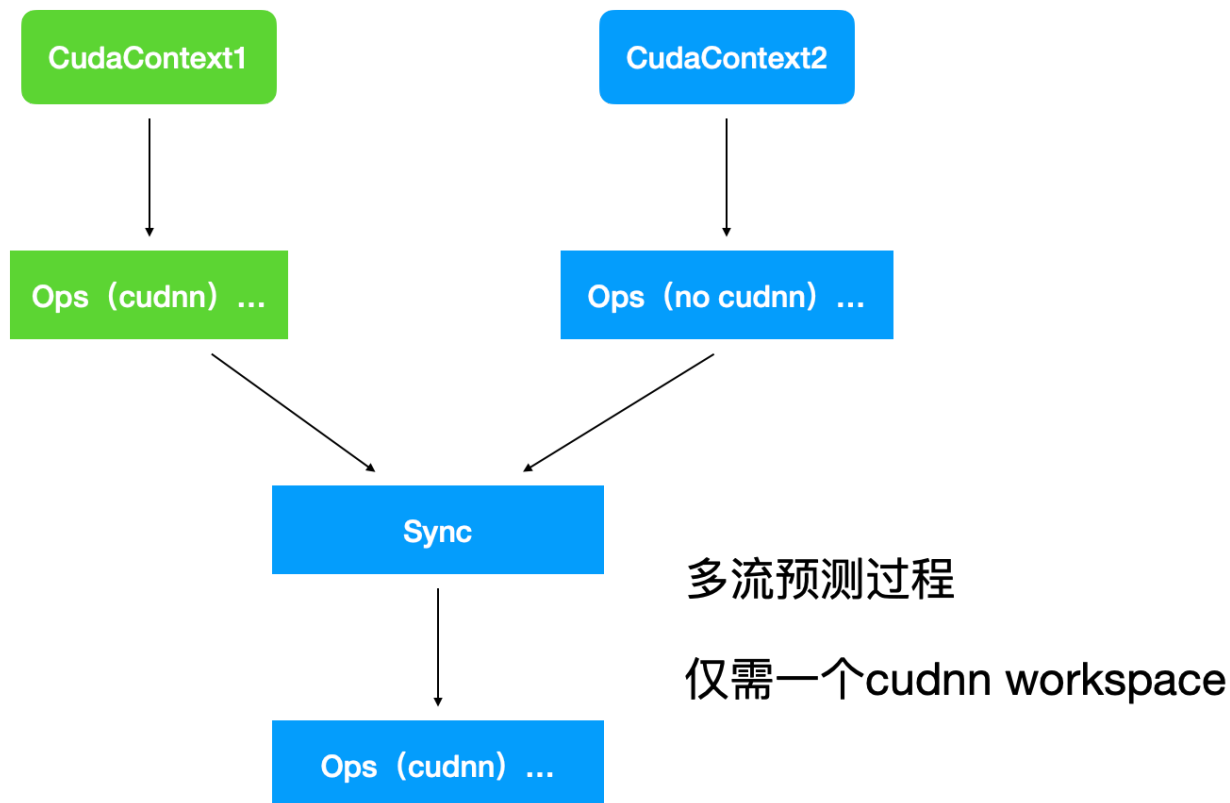
pten是算子库，因此其对DeviceContext的需求，即为算子对DeviceContext的需求，所以pten中DeviceContext仅应包含算子需要的接口，对于框架调度相关的接口不应进入算子库。

1.2、训练和推理重构对DeviceContext的需求

- 现有代码兼容：fluid训练和预测不应受到影响；
- 未来Lite能够接入；
- 目前现有实现存在以下问题，对于预测的需求无法满足。

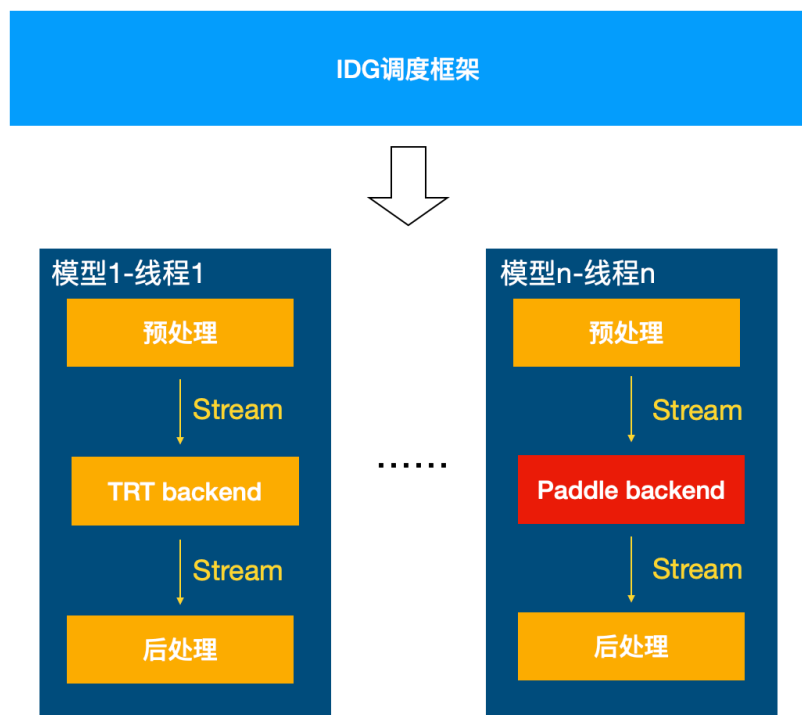
1.2.1 多个Context复用cudnn workspace场景（或复用其它资源）。

- 每个context维护自身的cudnn workspace，已支持；
- 多个context复用同一个cudnn workspace，无法支持；
- 以上两种情况共存。



1.2.2、kernel级别支持IO和exec stream的并发。

1.2.3、支持外部用户传流（IDG业务强需求），进一步拓展，支持外部用户传资源（cublasHandle, cudnnHandle）。



IDG的业务仍属于server范畴，其资源不是那么紧张；未来边缘端资源受限，预处理后处理过程中如果用到handle， cudnn workspace等资源，也有复用需求。

二、方案

2.1 方案综述

pten中放入DeviceContext的实现，但只放算子库需要的接口，不需要的接口保留在 platform::DeviceContext内，代码表述如下所示。

```
1. // pten
2. class pten::GPUContext : public pten::DeviceContext {
3. // ... 算子所需接口
4. };
5. // fluid
6. class platform::CUDADeviceContext : public pten::GPUContext {
7. // ... 框架相关(训练gc、预测线程流)所需接口等
8. };
```

2.2、现口

2.2.1、现有接口

1	接口	含义
2	CUDADeviceContext(CUDAPlace place);	带place构造函数
3	void Wait() const override	等待流中操作执行完成
4	Place GetPlace() const override	取出place
5	int GetComputeCapability() const	取出显卡固有属性
6	int GetMaxPhysicalThreadCount() const	取出显卡固有属性

7	<code>int GetSMCount() const</code>	取出显卡固有属性
8	<code>int GetMaxThreadsPerBlock() const</code>	取出显卡固有属性
9	<code>dim3 GetCUDAMaxGridDimSize() const</code>	取出显卡固有属性
10	<code>Eigen::GpuDevice* eigen_device() const</code>	获取gpu eigen计算接口
11	<code>void CublasCall(Callback&& callback) const</code>	对cublas调用的封装
12	<code>bool tensor_core_available() const</code>	是否支持tensor_core
13	<code>void TensorCoreCublasCallIfAvailable(Callback&& callback) const</code>	对cublas调用的封装
14	<code>cudaDnnHandle_t cudnn_handle() const</code>	获取cudnn handle
15	<code>cudaCublasHandle_t cublas_handle() const</code>	获取cublas handle
16	<code>CudnnWorkspaceHandle cudnn_workspace_handle() const</code>	为调用cudnn func预先分配空间，目前CudnnWorkSpace lazy初始化，且和Context绑定，无法满足预测多个context用cudnn workspace的场景
17	<code>cusolverDnHandle_t cusolver_dn_handle() const</code>	获取cusolver handle
18	<code>cudaStream_t stream() const</code>	获取stream，目前只支持取context中的默认stream，但运行在多个流（执行流和io流）上，需能够支持取出任意
19	<code>ncclComm_t nccl_comm() const</code>	获取nccl句柄
20	<code>void set_nccl_comm(ncclComm_t comm)</code>	设置nccl句柄
21	<code>void RecordEvent(cudaEvent_t ev, Callback callback) const</code>	目前在context中的stream资源上，设置event，无法满足kernel级别支持IO和exec stream的并发。
22	<code>void AddStreamCallback(Callback&& callback) const</code>	context中给默认stream加回调功能，无法满足kernel级持IO和exec stream的并发。
23	<code>void WaitStreamCallback() const</code>	——
	<code>void ResetDefaultContext(const</code>	没有地方用到

2024/3/27 17:16

Context 备份

24	<code>stream::Priority& priority)</code>	
25	<code>void ResetThreadContext(const stream::Priority& priority)</code>	预测为支持线程流，加的thread_local语义
26	<code>std::shared_ptr<CUDAContext> context() const</code>	获取底层封装的context，应该为Private或protected
27	<code>void SetThreadLocalStream(const gpuStream_t stream)</code>	外部用户传流，目前仅支持thread_local语义
28		

2.2.2、为支持预测场景，需新增的接口

1	接口	含义
2	<code>CUDAContext(cudaStream_t, cublasHandle_t, cudnnHandle_t, ...)</code>	外部资源构造
3	<code>cudaStream_t host_to_device_stream();</code> <code>cudaStream_t device_to_host_stream();</code> <code>... // device_to_device_stream()</code>	性能考虑，exec kernel实现，还的拷贝
4	<code>void SetAllocator(Allocator*);</code> <code>Allocator* GetAllocator();</code>	设置获取Alloca

三、方案论证

3.1、兼容fluid框架

DeviceContext相当于按照算子层和框架层需求进行了拆分，算子层依赖的接口在pten层，框架层依赖的接口在fluid层，天然兼容fluid。

3.2、对pten长久状态支持

pten::DeviceContext只开放算子开发需要的接口，对pten算子库开发无影响。

3.3、开发和兼容性代价

DeviceContext对用户接口保持不变，算子迁移无代价；如果接口的代码风格修改，对用户而言只有改接口名字的代价如 `ctx->GetPlace()` 替换为 `ctx->place()`，推全过程文档中添加说明即可。

将DeviceContext接口按照算子层和框架层进行拆分，分别放入pten和fluid内部，兼容fluid无代价。